

Graph Algorithms

Module -4

BCSE204L – Design and Analysis of Algorithms





Outline

- Bellman Ford Algorithm: Single Source Shortest Path Algorithm
- Floyd Warshall Algorithm: All-Pairs Shortest Path Algorithm



Bellman Ford Algorithm: SSSP

- Bellman Ford algorithm is used to find the shortest path from the source vertex to remaining all other vertices in the weighted graph. We can find an optimal solution to this problem using [dynamic programming](#).



Bellman Ford Algorithm: SSSP

- It is slower compared to Dijkstra's algorithm but it can handle **negative weights** also.
- If the graph contains negative **weight cycle** (edge sum of the cycle is negative), it is not possible to find the minimum path, because on every iteration of cycle gives a better result.
- Bellman Ford algorithm can detect a negative cycle in the graph but it cannot find the solution for such graphs.



Bellman Ford Algorithm: SSSP

- It is slower compared to Dijkstra's algorithm but it can handle **negative weights** also.
- If the graph contains negative **weight cycle** (edge sum of the cycle is negative), it is not possible to find the minimum path, because on every iteration of cycle gives a better result.
- Bellman Ford algorithm can detect a negative cycle in the graph but it cannot find the solution for such graphs.



Bellman Ford Algorithm: SSSP

- Like Dijkstra, this algorithm is also based on the principle of relaxation. The path is updated with better values until it reaches the optimal value.
- Dijkstra uses a priority queue to greedily select the closest vertex and perform relaxation on all its adjacent outgoing edges. By contrast, Bellman Ford relaxes all the edges $|V| - 1$ time. Bellman Ford runs in $O(|V| \cdot |E|)$ time.

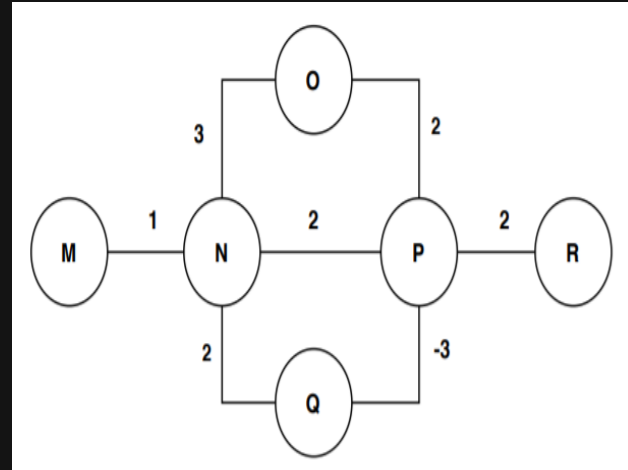


Bellman Ford Algorithm: SSSP

- **Why Are Negative Edges Important to Consider?**
- In graph theory, negative edges are more important as they can create a negative cycle in a given graph.
- Let's start with a simple weighted graph with a negative cycle and try to find out the shortest distance from one node to another:

Bellman Ford Algorithm: SSSP

- We're considering each node as a city. We want to **go to city R from city M**.
- There are three roads from the city M to reach the city R: **MNPR**, **MNQPR**, **MNOPR**. The road lengths are 5, 2, and 8.
- But, when we take a deeper look, we see that there's a negative cycle: **NQP**, which has a length of **-1**. So, **each time we cover the path NQP, our road length will decrease**.
- This leads to us not being able to get an exact answer on the shortest path since **repeating the road NQP infinite times would, by definition, be the least expensive route**.





Bellman Ford Algorithm: SSSP

- **How Bellman Ford's Algorithm works?**
- Bellman Ford algorithm works by overestimating the length of the path from the starting vertex to all other vertices.
- Then it iteratively relaxes those estimates by finding new paths that are shorter than the previously overestimated paths.
- By doing this repeatedly for all vertices, we can guarantee that the result is optimized.



Bellman Ford Algorithm: SSSP

Algorithm Bellman-Ford(weighted digraph (G,c), node s)

```
array dist[n] = { $\infty$ ,  $\infty$ ,...}  
dist[s]  $\leftarrow$  0  
FOR i from 0 to n - 1 DO  
  FOR x  $\in$  V (G) DO  
    FOR v  $\in$  V (G) DO  
      dist[v]  $\leftarrow$  min(dist[v], dist[x] + c(x,v))  
    END FOR  
  END FOR  
END FOR  
RETURN dist  
END
```

- Time complexity $\{ (n^3) \}$; unlike the Dijkstra's algorithm, it handles negative weight arcs (but no negative weight cycles making the SSSP senseless).



Bellman Ford Algorithm: SSSP

- Bellman Ford: all nodes at “level” $0, 1, \dots, n - 2$ in turn.
- Level of a node v – the minimum possible number of arcs in a minimum weight path to that node from the source s .
- **Theorem:** If a graph G contains no negative weight cycle, then after the i^{th} iteration of the outer for-loop, the element $dist[v]$ contains the minimum weight of a path to v for all nodes v with level at most i .

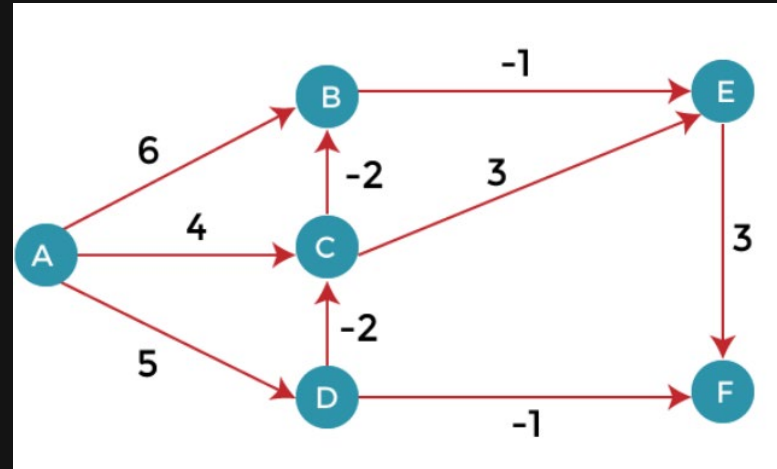


Bellman Ford Algorithm: SSSP

- Just as for Dijkstra's, the update ensures $dist[v]$ never increases.
- Induction by level i of the nodes"
- Base case: $i = 0$; the result is true due to initialization:
- $dist[s] = 0$; $dist[v] = \infty$; $v \in V \setminus s$.
- Induction hypothesis: $dist[v]$; $v \in V$, are true for $i - 1$.
- Induction step for a node v at level i :
 - Due to no negative cycles, a min-weight s to v path, γ , has i arcs.
 - If y is the last node before v and γ_1 the subpath to y , then $dist[y] \leq |\gamma_1|$ by the induction hypothesis.
 - Thus by the update rule:
 - $dist[v] \leq dist[y] + c(y, v) \leq |\gamma_1| + c(y, v) \leq |\gamma|$

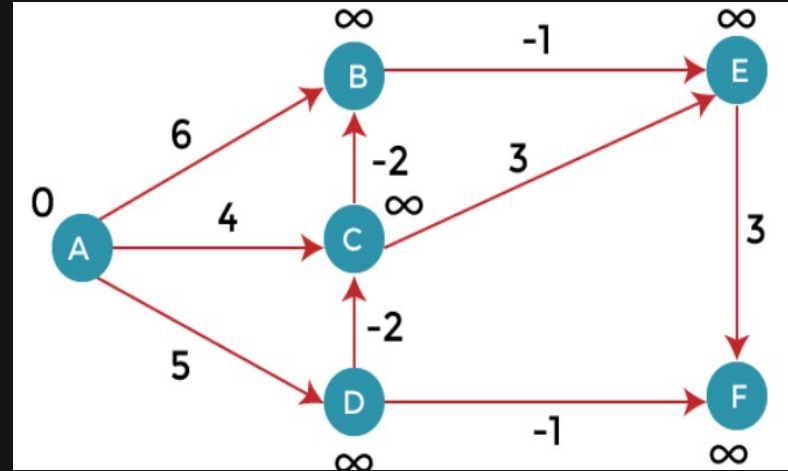
Bellman Ford Algorithm: SSSP

- Consider the graph
- **Relaxing means**
- *if ($dist[u] + c(u, v) < dist[v]$)
 $dist[v] = dist[u] + c(u, v)$*
- To find the shortest path of the above graph, the first step is note down all the edges which are given below:
- (A, B), (A, C), (A, D), (B, E), (C, E), (D, C), (D, F), (E, F), (C, B)



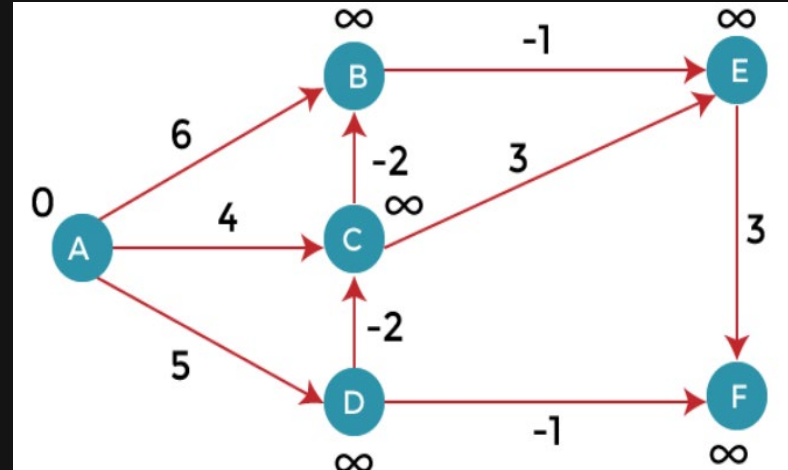
Bellman Ford Algorithm: SSSP

- Let's consider the source vertex as 'A'; therefore, the distance value at vertex A is 0 and the distance value at all the other vertices as infinity shown as below:



Bellman Ford Algorithm: SSSP

- First iteration
- Consider the edge (A, B) . Denote vertex ' A ' as ' u ' and vertex ' B ' as ' v '. Now use the relaxing formula:
 - $dist(u) = 0, dist(v) = \infty$
 - $c(u, v) = 6$, Since $(0 + 6)$ is less than ∞ , so update
$$dist(v) = dist(u) + c(u, v)$$
$$dist(v) = 0 + 6 = 6$$
- Therefore, the distance of vertex B is 6.





Bellman Ford Algorithm: SSSP

- First iteration – (A,C)

- $dist(u) = 0, dist(v) = \infty$
- $c(u, v) = 4$, Since $(0 + 4)$ is less than ∞ , so update

$$dist(v) = dist(u) + c(u, v)$$
$$dist(v) = 0 + 4 = 4$$

- Therefore, the distance of vertex C is 4.

- First iteration – (A,D)

- $dist(u) = 0, dist(v) = \infty$
- $c(u, v) = 5$, Since $(0 + 5)$ is less than ∞ , so update

$$dist(v) = dist(u) + c(u, v)$$
$$dist(v) = 0 + 5 = 5$$

- Therefore, the distance of vertex D is 5.



Bellman Ford Algorithm: SSSP

- First iteration – (B,E)

- $dist(u) = 6, dist(v) = \infty$
- $c(u, v) = -1$, Since $(6 - 1)$ is less than ∞ , so update

$$dist(v) = dist(u) + c(u, v)$$
$$dist(v) = 6 - 1 = 5$$

- Therefore, the distance of vertex E is 5.

- First iteration – (C,E)

- $dist(u) = 4, dist(v) = 5$
- $c(u, v) = 3$, Since $(4 + 3) > 5$, so **No update**
- Therefore, the distance of vertex E is 5.



Bellman Ford Algorithm: SSSP

- First iteration – (D,C)

- $dist(u) = 5, dist(v) = 4$
- $c(u, v) = -2$, Since $(5 - 2) < 4$, so update

$$dist(v) = dist(u) + c(u, v)$$
$$dist(v) = 5 - 2 = 3$$

- Therefore, the distance of vertex C is 3.

- First iteration – (D,F)

- $dist(u) = 5, dist(v) = \infty$
- $c(u, v) = -1$, Since $(5 - 1) < \infty$, so update

$$dist(v) = dist(u) + c(u, v)$$
$$dist(v) = 5 - 1 = 4$$

- Therefore, the distance of vertex F is 4.



Bellman Ford Algorithm: SSSP

- First iteration – (E,F)

- $dist(u) = 5, dist(v) = 4$
- $c(u, v) = 3$, Since $(5 + 3) > 4$, so **No update**
- Therefore, the distance of vertex **F** is **4**.

- First iteration – (C,B)

- $dist(u) = 3, dist(v) = 6$
- $c(u, v) = -2$, Since $(3 - 2) < 6$, so update
$$dist(v) = dist(u) + c(u, v)$$
$$dist(v) = 3 - 2 = 1$$
- Therefore, the distance of vertex **B** is **1**.

Bellman Ford Algorithm: SSSP

- First iteration

dist[v]						
$i =$	A	B	C	D	E	F
0	0	6 1	4 3	5	5	4
1						
2						
3						
4						
5						



Bellman Ford Algorithm: SSSP

- Second iteration ($i=1$)
- The edge is (A, B) . Since $(0 + 6) > 1$, so **No updation** in the vertex **B**.
- The edge is (A, C) . Since $(0 + 4) > 3$, so **No updation** in the vertex **C**.
- The edge is (A, D) . Since $(0 + 5) = 5$, so **No updation** in the vertex **D**.
- The edge is (B, E) . Since $(1 - 1) < 5$, so update $dist[e] = dist[b] + c(B, E) = 1 - 1 = 0$. The distance of the vertex **E** is 0.
- The edge is (C, E) . Since $(5 - 2) > 3$, so **No updation** in the vertex **E**.
- The edge is (D, C) . Since $(5 - 2) = 3$, so **No updation** in the vertex **C**.
- The edge is (D, F) . Since $(5 - 1) = 4$, so **No updation** in the vertex **F**.
- The edge is (E, F) . Since $(5 + 3) > 4$, so **No updation** in the vertex **F**.
- The edge is (C, B) . Since $(3 - 2) = 1$, so **No updation** in the vertex **B**.



Bellman Ford Algorithm: SSSP

- THIRD iteration ($i=2$)
 - The edge is (A, B) . so **No updation** in the vertex **B**.
 - The edge is (A, C) . so **No updation** in the vertex **C**.
 - The edge is (A, D) . so **No updation** in the vertex **D**.
 - The edge is (B, E) . so **No updation** in the vertex **E**.
 - The edge is (C, E) . so **No updation** in the vertex **E**.
 - The edge is (D, C) . so **No updation** in the vertex **C**.
 - The edge is (D, F) . so **No updation** in the vertex **F**.
 - The edge is (E, F) . so **No updation** in the vertex **F**.
 - The edge is (C, B) . so **No updation** in the vertex **B**.

Bellman Ford Algorithm: SSSP

- $i=1$

dist[v]						
$i =$	A	B	C	D	E	F
0	0	6 1	4 3	5	5	4
1	0	1	3	5	5 0	4
2	0	1	3	5	0	4
3						
4						
5						



Bellman Ford Algorithm: SSSP

- Time Complexity
 - Best Case Complexity $O(E)$
 - Average Case Complexity $O(V+E)$
 - Worst Case Complexity $O(V+E)$
- Space Complexity
 - And, the space complexity is $O(V)$.
- Bellman Ford's Algorithm Applications
 - For calculating shortest paths in routing algorithms
 - For finding the shortest path

Bellman Ford Algorithm: SSSP

- Bellman Ford does not produce a correct answer if the sum of the edges of a cycle is negative.
- For Example
- Bellman Ford can have only 3 iterations if there are 4 vertices.
- Consider the edges in the order

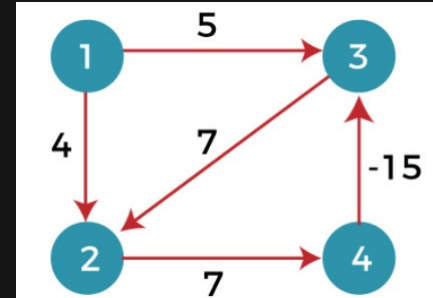
(1, 3),

(1, 2),

(2, 4),

(3, 2),

(4, 3)



Bellman Ford Algorithm: SSSP

- First iteration ($i=0$)
- The edge is $(1,3)$. Since $(0 + 5) > \infty$, so Update the vertex **3** as **5**.
- The edge is $(1,2)$. Since $(0 + 4) > \infty$, so Update the vertex **2** as **4**.
- The edge is $(2,4)$. Since $(4 + 7) > \infty$, so Update the vertex **4** as **11**.
- The edge is $(3,2)$. Since $(5 + 7) < 4$, so **No Update** the vertex **2**.
- The edge is $(4,3)$. Since $(11 - 15) > 5$, so Update the vertex **3** as **-4**.

dist[v]				
$i =$	1	2	3	4
0	0	4	5 -4	11
1				
2				
3				
4				

Bellman Ford Algorithm: SSSP

- Second iteration ($i=1$)
- The edge is $(1,3)$. Since $(0 + 5) > -4$, so **No Update** the vertex **3**.
- The edge is $(1,2)$. Since $(0 + 4) = 4$, so **No Update** the vertex **2**.
- The edge is $(2,4)$. Since $(4 + 7) = 11$, so **No Update** the vertex **4** as **11**.
- The edge is $(3,2)$. Since $(-4 + 7) > 4$, so Update the vertex **2** as **3**.
- **dist[2] is update hence check the edge (2,4)**
- The edge is $(2,4)$. Since $(3 + 7) > 11$, so Update the vertex **4** as **10**.
- The edge is $(4,3)$. Since $(10 - 15) > -4$, so Update the vertex **3** as **-5**.

$i =$	dist[v]			
	1	2	3	4
0	0	4	5 -4	11
1	0	4 3	-4 -5	11 10
2				
3				
4				

Bellman Ford Algorithm: SSSP

- Third iteration ($i=2$)
- The edge is $(1, 3)$. Since $(0 + 5) > -4$, so **No Update** the vertex **3**.
- The edge is $(1, 2)$. Since $(0 + 4) = 3$, so **No Update** the vertex **2**.
- The edge is $(3, 2)$. Since $(-5 + 7) > 3$, so Update the vertex **2** as **2**.
- The edge is $(2, 4)$. Since $(2 + 7) > 10$, so Update the vertex **4** as **9**.
- The edge is $(4, 3)$. Since $(9 - 15) > -4$, so Update the vertex **3** as **-6**.

	dist[v]			
$i =$	1	2	3	4
0	0	4	5 -4	11
1	0	4 3	-4 -5	11 10
2	0	3 2	-5 -6	10 9
3				
4				

Bellman Ford Algorithm: SSSP

- Since the graph contains 4 vertices, so according to the bellman ford algorithm, **there would be only 3 iterations**. If we try to perform 4th iteration on the graph, the distance of the vertices from the given vertex should not change. If the distance varies, it means that the bellman ford algorithm is not providing the correct answer.

$i =$	dist[v]			
	1	2	3	4
0	0	4	5 -4	11
1	0	4 3	-4 -5	11 10
2	0	3 2	-5 -6	10 9
3				
4				

Bellman Ford Algorithm: SSSP

- Fourth iteration ($i=3$)
- The edge is $(1, 3)$. Since $(0 + 5) > -4$, so **No Update** the vertex **3**.
- The edge is $(1, 2)$. Since $(0 + 4) = 3$, so **No Update** the vertex **2**.
- The edge is $(3, 2)$. Since $(-6 + 7) > 3$, so Update the vertex **2** as **1**.
- The edge is $(2, 4)$. Since $(1 + 7) > 10$, so Update the vertex **4** as **8**.
- The edge is $(4, 3)$. Since $(8 - 15) > -4$, so Update the vertex **3** as **-7**.

dist[v]				
$i =$	1	2	3	4
0	0	4	5 -4	11
1	0	4 3	-4 -5	11 10
2	0	3 2	-5 -6	10 9
3	0	2 1	-6 -7	9 8
4				

Bellman Ford Algorithm: SSSP

- In this case, the value of the vertex is updated. So, we conclude that the bellman ford algorithm does not work when the graph contains the negative weight cycle.
- Therefore, the value at vertex 2 is 1.

$i =$	dist[v]			
	1	2	3	4
0	0	4	5 -4	11
1	0	4 3	-4 -5	11 10
2	0	3 2	-5 -6	10 9
3	0	2 1	-6 -7	9 8
4				

Floyd Warshall Algorithm

All-Pairs Shortest Path Algorithm





Floyd Warshall Algorithm: APSP

- The Floyd-Warshall algorithm is a shortest path algorithm for [graphs](#).
- Like the [Bellman-Ford algorithm](#) or the [Dijkstra's algorithm](#), it computes the shortest path in a graph.
- However, Bellman-Ford and Dijkstra are both single-source, shortest-path algorithms.
- This means they only compute the shortest path from a single source.
- Floyd-Warshall, on the other hand, computes the shortest distances between every pair of vertices in the input graph.



Floyd Warshall Algorithm: APSP

- The Floyd-Warshall algorithm is an example of [dynamic programming](#).
- It breaks the problem down into smaller subproblems, then combines the answers to those subproblems to solve the big, initial problem.
- The idea is this: either the quickest path from A to C is the quickest path found so far from A to C, or it's the quickest path from A to B plus the quickest path from B to C.



Floyd Warshall Algorithm: APSP

- Floyd-Warshall is extremely useful in networking, similar to solutions to the shortest path problem.
- However, it is more effective at managing multiple stops on the route because it can calculate the shortest paths between all relevant nodes.
- In fact, one run of Floyd-Warshall can give you all the information you need to know about a static network to optimize most types of paths.
- It is also useful in computing matrix inversions.



Floyd Warshall Algorithm: APSP

- This algorithm works for both the directed and undirected weighted graphs.
- But, *it does not work for the graphs with negative cycles* (where the sum of the edges in a cycle is negative).
- Floyd-Warshall algorithm is also called as Floyd's algorithm, Roy-Floyd algorithm, Roy-Warshall algorithm, or WFI algorithm.



Floyd Warshall Algorithm: APSP

- Floyd's algorithm
- For a graph with N vertices:
 1. Initialize the shortest paths between any 2 vertices with Infinity.
 2. Find all pair shortest paths that use 0 intermediate vertices, then find the shortest paths that use 1 intermediate vertex and so on.. until using all N vertices as intermediate nodes.
 3. Minimize the shortest paths between any 2 pairs in the previous operation.
 4. For any 2 vertices (i,j) , one should actually minimize the distances between this pair using the first K nodes, so the shortest path will be: $\min(A[i][k]+A[k][j], A[i][j])$.



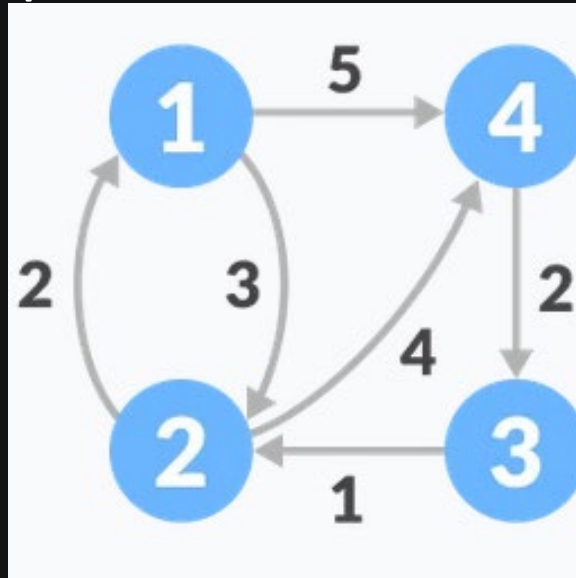
Floyd Warshall Algorithm: APSP

- Floyd's algorithm
- $A[i][k]$ represents the shortest path that only uses the first K vertices, $A[k][j]$ represents the shortest path between the pair k, j . As the shortest path will be a concatenation of the shortest path from i to k , then from k to j .

```
FOR (int k = 1; k <= n; k++)  
{  
  FOR (int i = 1; i <= n; i++)  
  {  
    FOR (int j = 1; j <= n; j++)  
    {  
      A[i][j] = min( A[i][j], A[i][k] +  
                    A[k][j] );  
    }  
  }  
}
```

Floyd Warshall Algorithm: APSP

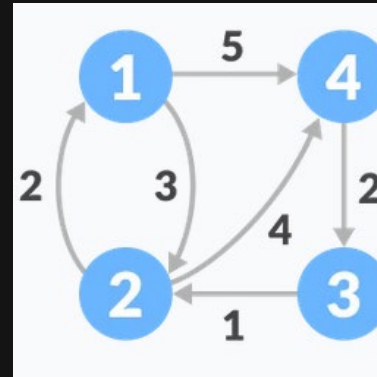
- How Floyd's algorithm Works?
- Consider the graph with 4 vertices:



Floyd Warshall Algorithm: APSP

- Step 1:
- Create a matrix A^0 of dimension $n * n$ where n is the number of vertices. The row and the column are indexed as i and j respectively. i and j are the vertices of the graph.
- Each cell $A[i][j]$ is filled with the distance from the i th vertex to the j th vertex. If there is no path from i th vertex to j th vertex, the cell is left as infinity.

$$A^0 = \begin{bmatrix} 0 & 3 & \infty & 5 \\ 2 & 0 & \infty & 4 \\ \infty & 1 & 0 & \infty \\ \infty & \infty & 2 & 0 \end{bmatrix}$$





Floyd Warshall Algorithm: APSP

- Step 2:
- Now, create a matrix A^1 using matrix A^0 . The elements in the first column and the first row are left as they are. The remaining cells are filled in the following way.
- Let k be the intermediate vertex in the shortest path from source to destination. In this step, k is the first vertex. $A[i][j]$ is filled with $(A[i][k] + A[k][j])$ if $(A[i][j] > A[i][k] + A[k][j])$.
- That is, if the direct distance from the source to the destination is greater than the path through the vertex k , then the cell is filled with $A[i][k] + A[k][j]$.
- In this step, k is vertex 1. We calculate the distance from source vertex to destination vertex through this vertex k .



Floyd Warshall Algorithm: APSP

- Step 2:

- $A^1 = \begin{bmatrix} 0 & 3 & \infty & 5 \\ 2 & 0 & & \\ \infty & & 0 & \\ \infty & & & 0 \end{bmatrix} \rightarrow \begin{bmatrix} 0 & 3 & \infty & 5 \\ 2 & 0 & \infty & 4 \\ \infty & 1 & 0 & \infty \\ \infty & \infty & 2 & 0 \end{bmatrix}$

- For $A^1[2, 4]$, the direct distance from vertex 2 to 4 is 4 and the sum of the distance from vertex 2 to 4 through vertex (ie. from vertex 2 to 1 and from vertex 1 to 4) is 7. Since $4 < 7$, $A^0[2, 4]$ is filled with 4.



Floyd Warshall Algorithm: APSP

- Step 3:
- Similarly, A^2 is created using A^1 . The elements in the second column and the second row are left as they are.

$$\bullet A^2 = \begin{bmatrix} 0 & 3 & \infty & 4 \\ 2 & 0 & \infty & 4 \\ & 1 & 0 & \\ & \infty & & 0 \end{bmatrix} \rightarrow \begin{bmatrix} 0 & 3 & 7 & 5 \\ 2 & 0 & \infty & 4 \\ 3 & 1 & 0 & 5 \\ \infty & \infty & 2 & 0 \end{bmatrix}$$



Floyd Warshall Algorithm: APSP

- Step 4:
- Similarly, A^3 is created using A^2 . The elements in the second column and the second row are left as they are.

$$\bullet A^3 = \begin{bmatrix} 0 & & 7 & \\ & 0 & \infty & \\ 3 & 1 & 0 & 5 \\ & & 2 & 0 \end{bmatrix} \rightarrow \begin{bmatrix} 0 & 3 & 7 & 5 \\ 2 & 0 & \infty & 4 \\ 3 & 1 & 0 & 5 \\ 5 & 3 & 2 & 0 \end{bmatrix}$$



Floyd Warshall Algorithm: APSP

- Step 5:
- Similarly, A^4 is created using A^3 . The elements in the second column and the second row are left as they are.

$$\bullet A^4 = \begin{bmatrix} 0 & & & 5 \\ & 0 & & 4 \\ & & 0 & 5 \\ 5 & 3 & 2 & 0 \end{bmatrix} \rightarrow \begin{bmatrix} 0 & 3 & 7 & 5 \\ 2 & 0 & 6 & 4 \\ 3 & 1 & 0 & 5 \\ 5 & 3 & 2 & 0 \end{bmatrix}$$



Floyd Warshall Algorithm: APSP

- **Time Complexity:**
- There are three loops. Each loop has constant complexities. So, the time complexity of the Floyd-Warshall algorithm is $O(n^3)$.
- **Space Complexity:**
- The space complexity of the Floyd-Warshall algorithm is $O(n^2)$.



Floyd Warshall Algorithm: APSP

- Applications:
- To find the shortest path in a directed graph
- To find the transitive closure of directed graphs
- To find the Inversion of real matrices
- For testing whether an undirected graph is bipartite

Module 4

Graph algorithm

BCSE204L – Design and Analysis of Algorithm

